Автономная некоммерческая профессиональная образовательная организация «Хекслет колледж»

КУРСОВОЙ ПРОЕКТ

по дисциплине Инструментальные средства разработки

Тема: «Трекер чтения книг»

Специальность 09.02.07 «Информационные системы и программирование»

Руководитель _____ Э.И. Шарипова
подпись И.О.Фамилия

Студент Мухин Сергей Олегович Группа 15.ИСиП.24.ОФ.О.2.ХК
Фамилия И.О.

Работа выполнена с оценкой _____

Москва – 2026 года

Автономная некоммерческая профессиональная образовательная организация

«ХЕКСЛЕТ КОЛЛЕДЖ»

(АНПОО «Хекслет колледж»)

ЗАДАНИЕ

для выполнения курсового проекта

Профессиональный модуль: ПМ.02 Осуществление интеграции программных модулей, МДК.02.02 Инструментальные средства разработки

Специальность: 09.02.07 Информационные системы и программирование

Группа 15.ИСиП.24.ОФ.О.2.ХК/Мухин Сергей Олегович

Курсовой проект: «Трекер чтения книг»

Дата выдачи задания «18» февраля 2026 г.

Проект должен быть сдан не позднее «17» апреля 2026 г.

Содержание курсового проекта

Введение

1. Аналитическая часть

    1.1 Описание предметной области

    1.2 Анализ существующих аналогов

    1.3 Требования к разрабатываемой информационной системе

        1.3.1 Функциональные требования

        1.3.2 Нефункциональные требования

    1.4 Обоснование выбора методики, технологии и инструментальных средств проектирования и разработки

2. Проектная часть

    2.1 Проектирование структур данных

    2.2 Реализация базового функционала

    2.3 Работа с массивами и алгоритмами

    2.4 Взаимодействие с DOM

2.5 Локальное хранение данных

2.6 Валидация и обработка ошибок

3. Реализация

3.1 Реализация основных функций веб-приложения/информационной системы

3.2 Тестирование веб-приложения/информационной системы

Заключение

Приложения

Руководитель курсового проекта _____ / Э. И. Шарипова /

Содержание

Содержание 4
Введение 5
1. АНАЛИТИЧЕСКАЯ ЧАСТЬ 7
1.1. Описание предметной области 7
1.2. Анализ существующих аналогов 8
1.3 Требования к разрабатываемой информационной системе 9
1.3.1 Функциональные требования 9
1.3.2 Нефункциональные требования 10
1.4 Обоснование выбора технологического стека и инструментов разработки 10
2. ПРАКТИЧЕСКАЯ ЧАСТЬ 13
2.1 Проектирование архитектуры и структуры данных 13
2.2 Реализация базового функционала 14
2.3 Работа с массивами и алгоритмами 14
2.4 Взаимодействие с DOM 15
2.5 Локальное хранение данных 17
2.6 Валидация пользовательского ввода 17
3. РЕАЛИЗАЦИЯ 18
3.1 Реализация основных функций веб-приложения 18
Архитектура приложения: 18
3.2 Тестирование веб-приложения 20
Заключение 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 24
Приложение 26

Введение

В современном мире информационные технологии играют ключевую роль в организации повседневной деятельности человека. Постоянный рост объёма информации, высокая интенсивность интеллектуального труда и необходимость формирования устойчивой привычки к систематическому чтению обусловливают востребованность цифровых инструментов для работы с книгами. Одним из наиболее удобных и наглядных решений являются приложения формата «трекер чтения книг» (BookTracker), позволяющие фиксировать прочитанные произведения, отслеживать прогресс по текущим книгам, ставить цели по количеству страниц или времени и тем самым повышать личную эффективность в самообразовании.

Актуальность разработки подобных приложений определяется широким распространением веб-технологий и их доступностью. Использование языков разметки и программирования, таких как HTML, CSS и JavaScript, позволяет создавать интерактивные и удобные пользовательские интерфейсы, работающие непосредственно в браузере без необходимости установки дополнительного программного обеспечения. Трекер чтения, реализованный в виде веб-приложения, даёт пользователю возможность в любой момент добавить новую книгу, оценить свой прогресс и наглядно посмотреть свою статистику виде таблицы.

Целью данной курсовой работы является разработка веб-приложения «Трекер чтения книг» с использованием HTML, CSS и JavaScript. Для достижения поставленной цели необходимо решить следующие задачи: изучить основы веб-разработки, проанализировать существующие решения в области цифрового учёта чтения (например: Goodreads, Litsy), спроектировать структуру приложения, реализовать пользовательский интерфейс и функциональность добавления книг, удаления записей, редактирования текущего прогресса (количество прочитанных страниц или процент завершения), а также провести тестирование разработанного продукта.

Объектом исследования является процесс управления личной библиотекой и отслеживания читательского прогресса с использованием веб-приложений, а предметом — методы и средства разработки клиентских приложений на базе современных веб-технологий для ведения читательского дневника.

Практическая значимость работы заключается в создании простого и эффективного инструмента для учёта прочитанной литературы, формирования устойчивой привычки к чтению и визуализации прогресса, который может быть использован в повседневной жизни, а также в закреплении теоретических знаний и приобретении практических навыков разработки веб-приложений.

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание предметной области

Чтение книг – это не просто хобби, а системная деятельность, требующая организации и самодисциплины. В современной практике управления личным временем (тайм-менеджменте) выделяют следующие ключевые аспекты, связанные с чтением:

- Планирование чтения – определение списка книг для прочтения с приоритизацией.
- Учёт времени и объёма – фиксация количества прочитанных страниц за каждую сессию чтения.
- Отслеживание прогресса – визуализация текущего состояния по каждой книге в процентном соотношении.
- Мотивация – создание системы поощрений за достижение целей (ежедневных и накопительных).
- Анализ эффективности – расчёт скорости чтения, регулярности занятий и серий (streak).

Ключевыми сущностями в данной предметной области являются:

- Книга (Book) – объект с атрибутами: уникальный идентификатор, название, автор, общее количество страниц, прочитано страниц, текущий статус (читаю/закончил/отложил), дата начала чтения, дата последнего обновления.
- История чтения (ReadingHistory) – массив записей, каждая из которых содержит дату и количество прочитанных за этот день страниц.
- Дневная цель (DailyGoal) – плановое количество страниц для прочтения за один день.
- Достижения (Badges) – набор бейджей, выдаваемых пользователю при выполнении определённых условий (первая завершённая книга, недельная серия, тысяча страниц).

Актуальность разработки легковесного, не требующего серверной части, веб-приложения обосновывается следующими факторами:

- Отсутствие необходимости регистрации и входа в систему – пользователь начинает работу мгновенно.
- Полный контроль пользователя над своими данными – информация не передаётся на сторонние серверы.
- Возможность использования без подключения к интернету после однократной загрузки страницы.
- Простота развёртывания – достаточно открыть HTML-файл в любом современном браузере.

1.2. Анализ существующих аналогов

Был проведён сравнительный анализ популярных веб-приложений и сервисов для учёта чтения. В качестве основных критериев сравнения выбраны: необходимость регистрации, наличие серверной части, отслеживание прогресса по страницам, дневные цели, система достижений, графики прогресса, работа офлайн и мотивационные цитаты.

Goodreads – крупнейшая социальная сеть для читателей. Основные недостатки: обязательная регистрация, отсутствие детального отслеживания прогресса по страницам (только статус «прочитано/читаю/хочу прочитать»), нет дневных целей и графиков, работа только при наличии интернета.

Livelib – российский аналог Goodreads. Недостатки: также требует регистрации, система достижений представлена только оценками и рецензиями, нет инструментов для ежедневной мотивации, отсутствует визуализация прогресса в виде графиков.

Выводы из анализа:

- Существующие сервисы требуют регистрации и постоянного подключения к интернету.
- Большинство аналогов не предоставляют детального отслеживания прогресса по страницам, ограничиваясь статусной моделью.
- Функции мотивации (ежедневные цели, достижения, визуализация прогресса) реализованы слабо или отсутствуют полностью.

- Легковесных клиентских решений на чистом JavaScript для трекинга чтения практически нет.

Таким образом, разрабатываемый трекер обладает уникальным набором функций, ориентированных именно на мотивацию и детальный учёт, что делает его конкурентоспособным решением в своей нише.

1.3 Требования к разрабатываемой информационной системе

1.3.1 Функциональные требования

Добавление новой книги. Система должна предоставлять форму с полями: название книги, автор, общее количество страниц. Все поля обязательны для заполнения. После добавления книга автоматически получает статус «Читаю».

Просмотр списка всех книг. Система должна отображать список всех добавленных книг в виде карточек или строк с информацией: название, автор, текущий прогресс в страницах и процентах, статус.

Отметка прогресса чтения. Пользователь должен иметь возможность отметить количество прочитанных страниц для любой книги. Система автоматически обновляет прогресс и проверяет, не завершена ли книга.

Изменение статуса книги. Пользователь может вручную изменить статус книги на один из трёх: «Читаю», «Закончил», «Отложил».

Установка дневной цели. Пользователь может установить ежедневную цель – количество страниц, которое он планирует читать в день.

Отображение прогресса выполнения цели. Система должна визуализировать прогресс выполнения дневной цели с помощью прогресс-бара и текстового индикатора.

Расчёт и отображение статистики. Система автоматически рассчитывает и отображает: количество завершённых книг, общее количество прочитанных страниц, среднюю скорость чтения за последние 7 дней, текущую серию (непрерывных дней чтения).

Визуализация прогресса за 7 дней. Система должна строить линейный график, показывающий количество прочитанных страниц за каждый из последних 7 дней.

Система достижений (бейджи). Система должна выдавать достижения при выполнении условий: завершение первой книги, непрерывное чтение в течение 7 дней,

достижение суммарного объёма 1000 страниц. Заблокированные достижения отображаются серым цветом, разблокированные – цветными.

Мотивационные цитаты. При каждом обновлении страницы или перезагрузке приложения должна отображаться случайная цитата о чтении.

Фильтрация книг по статусу. Пользователь должен иметь возможность фильтровать список книг по статусу: все книги, только читаемые, только завершённые, только отложенные.

Валидация вводимых данных. Система должна проверять корректность вводимых пользователем данных: название и автор не могут быть пустыми, количество страниц – положительное число.

1.3.2 Нефункциональные требования

Полная клиентская реализация. Приложение работает полностью в браузере без бэкенда. Все вычисления и хранение данных производятся на стороне клиента.

Локальное хранение данных. Данные пользователя (книги, история чтения, настройки) сохраняются в localStorage браузера и восстанавливаются после перезагрузки страницы.

Адаптивный пользовательский интерфейс. Интерфейс должен корректно отображаться на экранах с разрешением от 320 пикселей (мобильные устройства) и выше.

Кросс-браузерная совместимость. Приложение должно корректно работать в современных версиях браузеров: Yandex, Google Chrome, Mozilla Firefox, Microsoft Edge.

Удобство использования. Интерфейс должен быть интуитивно понятным: все элементы управления имеют понятные подписи, статусы выделяются цветами, действия сопровождаются уведомлениями.

Модульность кода. Исходный код должен быть разделён на логические модули для облегчения поддержки и дальнейшего развития.

1.4 Обоснование выбора технологического стека и инструментов разработки

Обоснование выбора стека технологий:

- HTML – обеспечивает семантическую разметку страницы, поддержку форм, модальных окон и элемента canvas для отрисовки графиков. Современные теги и атрибуты позволяют создавать структурированный и доступный интерфейс.
- CSS – предоставляет мощные средства для стилизации: Flexbox и Grid для построения адаптивных сеток, градиенты и тени для улучшения визуального восприятия. Медиа-запросы обеспечивают корректное отображение на устройствах с разным размером экрана.
- JavaScript – современный стандарт языка, поддерживающий стрелочные функции, модули. Чистый JavaScript без дополнительных фреймворков обеспечивает лёгкость приложения и отсутствие внешних зависимостей (за исключением Chart.js).
- Chart.js – легковесная библиотека для построения графиков, основанная на HTML Canvas. Не требует бэкенда, проста в использовании, предоставляет хорошие возможности для кастомизации.

Преимущества использования localStorage для данного типа приложения:

- Объём данных пользователя не превышает лимиты localStorage (5-10 МБ), что делает этот механизм идеальным.
- Нет необходимости в аутентификации, сложной инфраструктуре и серверном хранении.
- Данные сохраняются автоматически после каждого изменения, исключая риск потери информации.

Выбор методологии разработки:

- Модульный подход – код разделён на функциональные модули: constants.js для хранения констант и перечислений, utils.js для вспомогательных функций, data.js для управления данными и localStorage, bookManager.js для основной бизнес-логики и взаимодействия с DOM. Такое разделение упрощает отладку, тестирование и дальнейшее развитие проекта.

Инструменты:

- VS Code – основная среда разработки с поддержкой подсветки синтаксиса, автодополнения и интеграции с Git.

- Live Server – расширение для VS Code, обеспечивающее автоматическую перезагрузку страницы при изменениях кода.
- Git – система контроля версий для отслеживания изменений в процессе разработки.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Проектирование архитектуры и структуры данных

В ходе проектирования была разработана следующая структура объекта, представляющего книгу в системе:

- id (number) – уникальный числовой идентификатор, автоматически задаются при добавлении новой книги.
- title (string) – название книги, обязательное поле, максимальная длина 100 символов.
- author (string) – автор книги, обязательное поле, максимальная длина 100 символов.
- totalPages (number) – общее количество страниц в книге, положительное целое число от 1 до 10000.
- readPages (number) – количество уже прочитанных страниц, инициализируется нулём.
- status (string) – текущий статус книги, принимает одно из трёх значений: 'reading' (читаю), 'completed' (закончил), 'postponed' (отложил).
- startDate (string) – дата добавления книги в формате YYYY-MM-DD.
- lastUpdate (string) – дата и время последнего обновления прогресса

Структура истории чтения (ReadingHistory):

История чтения представляет собой массив записей, каждая из которых содержит:

- date (string) – дата чтения в формате YYYY-MM-DD.
- pages (number) – количество страниц, прочитанных в этот день.

Структура настроек в виде отдельных переменных:

- dailyGoal (number) – дневная цель в страницах, значение по умолчанию 30.
- nextBookId (number) – следующий доступный идентификатор для новой книги, инициализируется единицей.

Для хранения данных используется объект localStorage браузера. Определены следующие ключи:

- bookTracker_books – хранит сериализованный в JSON массив книг.
- bookTracker_history – хранит сериализованный в JSON массив истории чтения.
- bookTracker_dailyGoal – хранит число – значение дневной цели.
- bookTracker_nextId – хранит число – следующий доступный идентификатор книги.

2.2 Реализация базового функционала

Базовый функционал:

Create (создание) – функция addBook(title, author, totalPages) создаёт новый объект книги с начальными значениями (прочитано 0 страниц, статус «Читаю», текущая дата), добавляет его в массив books, сохраняет изменения и перерисовывает интерфейс.

Read (чтение) – функция renderBooksList() отображает текущий список книг в интерфейсе. При этом учитывается активный фильтр, и отображаются только книги, соответствующие выбранному статусу.

Update (обновление) – две основные функции обновления: updateProgress(bookId, pagesRead) для добавления прочитанных страниц и changeBookStatus(bookId, newStatus) для изменения статуса книги. Обе функции сохраняют изменения и обновляют интерфейс.

В модуле utils.js реализованы функции валидации:

- validateString(str, fieldName) – проверяет, что строка не пуста и не превышает 100 символов.
- validatePages(pages) – проверяет, что введённое значение является положительным числом и не превышает 10000 страниц.

2.3 Работа с массивами и алгоритмами

В процессе разработки были реализованы следующие алгоритмы обработки данных:

Алгоритм фильтрации книг по статусу:
Применяется метод filter к массиву книг. Если текущий фильтр установлен в значение 'all',

возвращается весь массив. В противном случае возвращаются только те книги, у которых поле status совпадает со значением фильтра.

Алгоритм расчёта серии (streak) непрерывного чтения: Функция calculateStreak() определяет, сколько дней подряд пользователь читал книги. Алгоритм включает следующие шаги:

- Проверка наличия записей в истории чтения.
- Сортировка записей по дате в порядке убывания.
- Получение текущей даты с обнулением времени.
- Последовательный перебор записей и сравнение разницы в днях с текущим значением серии.

Алгоритм расчёта средней скорости чтения: Функция calculateAverageSpeed() использует метод slice для получения последних 7 записей истории, затем метод reduce для суммирования страниц и деление на количество записей. Если записей меньше 7, используется доступное количество.

Алгоритм расчёта общего количества прочитанных страниц: Функция getTotalPagesRead() применяет метод reduce к массиву книг, суммируя значение readPages каждой книги.

Алгоритм выбора случайной цитаты: Функция updateDailyQuote() использует Math.random() для генерации случайного индекса в диапазоне от 0 до длины массива цитат, после чего отображает выбранную цитату.

Алгоритм расчёта процента прочтения книги: Функция calculatePercent(readPages, totalPages) вычисляет процентное соотношение прочитанных страниц к общему количеству. При делении на ноль возвращается 0. Результат округляется до целого числа.

2.4 Взаимодействие с DOM

Проектирование пользовательского интерфейса:

- Интерфейс приложения разделён на две основные колонки, что обеспечивает удобное расположение элементов управления и отображения данных.

Левая колонка содержит:

- Форму добавления книги с полями ввода и кнопкой отправки.
- Блок установки дневной цели с полем ввода, кнопкой установки, прогресс-баром и текстовым индикатором.
- Блок мотивационных сообщений и цитат.
- Блок достижений с тремя бейджами, отображающимися в заблокированном или разблокированном состоянии.

Правая колонка содержит:

- Панель фильтров с кнопками для отображения книг по статусам.
- Таблицу (визуально) со списком книг, где каждая строка содержит информацию о книге, прогрессе, статусе и элементах управления.
- Блок статистики с четырьмя карточками, отображающими ключевые метрики.
- График прогресса чтения за последние 7 дней.

Адаптивный дизайн:

Использование CSS Grid позволяет интерфейсу адаптироваться под разные размеры экрана. При ширине экрана менее 900 пикселей срабатывает медиа-запрос, который переключает расположение колонок с горизонтального на вертикальное. Также предусмотрена горизонтальная прокрутка для таблицы книг на узких экранах.

Функции динамического обновления интерфейса:

- renderAll() – основная функция, выполняющая полную перерисовку всех компонентов интерфейса.
- renderBooksList() – перерисовывает только таблицу книг с учётом текущего фильтра.
- updateStatistics() – обновляет числовые значения в карточках статистики.
- renderDailyGoalProgress() – обновляет прогресс-бар и текстовый индикатор дневной цели.
- updateChart() – полностью перестраивает график прогресса на основе актуальных данных.
- updateFilterButtons() – изменяет стиль активной кнопки фильтра.

2.5 Локальное хранение данных

Разработка модуля для работы с localStorage (В модуле data.js реализованы две ключевые функции):

- loadDataFromStorage() – выполняется при загрузке приложения. Функция пытается получить данные из localStorage по всем определённым ключам. При успешном получении данные парсятся из JSON и присваиваются глобальным переменным. При отсутствии данных переменные остаются с значениями по умолчанию. Дополнительно функция корректирует значение nextBookId, если в загруженных книгах встречаются идентификаторы, превышающие текущее значение.
- saveDataToStorage() – вызывается после каждого изменения данных. Функция сериализует массивы книг и истории в JSON, преобразует числовые значения в строки и сохраняет их в localStorage.

Механизм автоматического сохранения:

Сохранение данных происходит автоматически при выполнении следующих действий:

- Добавление новой книги.
- Обновление прогресса чтения.
- Изменение статуса книги.
- Установка новой дневной цели.

Такой подход гарантирует, что данные пользователя не будут потеряны при случайной перезагрузке страницы или закрытии браузера.

2.6 Валидация пользовательского ввода

Реализация проверки корректности ввода данных:

- Проверка на пустоту полей «Название» и «Автор» – пользователь не может добавить книгу без указания названия или автора.
- Проверка длины строк – названия и имена авторов длиннее 100 символов отклоняются.
- Проверка количества страниц – значение должно быть положительным числом от 1 до 10000.

- Проверка при отметке прогресса – количество страниц должно быть положительным числом; сумма прочитанных страниц не может превышать общее количество страниц книги.

Обработка возможных ошибок:

- Граничные случаи – при отсутствии книг в списке отображается сообщение «Нет книг». При отсутствии истории чтения график отображает нулевые значения. При неустановленной дневной цели используется значение по умолчанию 30 страниц.
- Валидация прогресса – если пользователь пытается отметить больше страниц, чем осталось до завершения книги, система отображает сообщение об ошибке и не сохраняет изменения.
- Информирование пользователя – все ошибки и важные события сопровождаются сообщениями в блоке мотивационных сообщений, который выделяется цветом для привлечения внимания.

3. РЕАЛИЗАЦИЯ

3.1 Реализация основных функций веб-приложения

Архитектура приложения:

Проект имеет следующую файловую структуру:

- index.html – основной HTML-файл, содержащий разметку всех элементов интерфейса и подключающий внешние ресурсы (стили, скрипты, библиотеку Chart.js).
- style.css – файл каскадных таблиц стилей, определяющий внешний вид всех элементов, включая адаптивную сетку, цветовую схему, анимации и состояния элементов.
- js/constants.js – модуль с константами: статусы книг, текстовые представления, CSS-классы для статусов, массив мотивационных цитат, ключи для localStorage.
- js/utils.js – модуль со вспомогательными функциями: получение текущей даты, валидация строк и чисел, расчёт процента выполнения.

- js/data.js – модуль, отвечающий за хранение и управление данными: глобальные переменные, функции загрузки и сохранения в localStorage, инициализация демонстрационных данных.
- js/bookManager.js – основной модуль, содержащий бизнес-логику приложения: функции добавления книг, обновления прогресса, изменения статусов, расчёта статистики, отрисовки интерфейса, обработчики событий.

Интеграция всех модулей:

Загрузка модулей происходит в определённом порядке в файле index.html:

- Подключение Chart.js из CDN.
- Подключение constants.js – необходим для определения констант, используемых во всех остальных модулях.
- Подключение utils.js – зависит только от констант.
- Подключение data.js – использует константы и утилиты.
- Подключение bookManager.js – использует все предыдущие модули.
- Атрибут defer у всех тегов script обеспечивает выполнение скриптов после полной загрузки HTML-документа, что гарантирует доступность всех DOM-элементов.

Ключевые функции приложения:

Функция добавления книги – при отправке формы срабатывает обработчик, который сначала проводит валидацию всех полей с помощью функций из utils.js. Если валидация пройдена успешно, создаётся объект новой книги с уникальным идентификатором, нулевым прогрессом, статусом «Читаю», текущей датой начала и временной меткой. Объект добавляется в массив books, вызывается сохранение в localStorage и полная перерисовка интерфейса.

Функция отметки прогресса – открывает модальное окно, где пользователь вводит количество прочитанных страниц. После подтверждения функция проверяет, не превышает ли новый общий прогресс общее количество страниц. Если всё корректно, обновляется значение readPages книги, обновляется временная метка, проверяется, не завершена ли книга (при достижении 100% прогресса статус автоматически меняется на «Закончил»), добавляется запись в историю чтения за текущий день, сохраняются изменения, перерисовывается интерфейс и проверяется выполнение дневной цели.

Функция расчёта серии (streak) – реализует алгоритм подсчёта непрерывных дней чтения. Функция создаёт копию массива истории, сортирует её по убыванию дат, затем последовательно проверяет, были ли чтения в каждый из последних дней. Если за текущий день было прочитано хотя бы 1 страница, серия продолжается. Алгоритм устойчив к пропускам дней и корректно обнуляет серию при перерыве.

Функция построения графика – собирает данные за последние 7 дней: для каждого дня создаётся метка (день и месяц) и значение (количество страниц из истории или 0). Затем создаётся новый график типа «линия» с настройками цвета, заливки и сглаживания.

Система достижений – функция checkBadges() вызывается при завершении книги или обновлении статистики. Она проверяет три условия: количество завершённых книг (для бейджа «Первая книга»), текущую серию (для бейджа «7 дней чтения»), общее количество прочитанных страниц (для бейджа «1000 страниц»). При выполнении условия соответствующий бейдж теряет класс locked, меняет цвет фона и выводит мотивационное сообщение.

3.2 Тестирование веб-приложения

Проведение модульного тестирования ключевых функций:

В ходе разработки было проведено ручное модульное тестирование следующих функций:

- **Функция** validatePages – проверялась на различных входных данных: отрицательные числа (возвращает ошибку), ноль (ошибка), строки (ошибка), числа в допустимом диапазоне (успех), числа свыше 10000 (ошибка).
- **Функция** calculatePercent – тестировалась на парах (0, 100) → 0%, (50, 100) → 50%, (100, 100) → 100%.
- **Функция** calculateStreak – тестировалась на наборе тестовых данных: пустая история → 0, один день чтения → 1, три дня подряд → 3, день чтения, затем пропуск, затем чтение → серия обнуляется.
- **Функция** addBook – проверялась на корректность создания объекта: все поля заполнены правильно, идентификатор инкрементируется, дата установлена корректно.
- **Функция** updateProgress – проверялась на граничные случаи: добавление страниц к несуществующей книге (отказ), превышение общего количества

страниц (отказ и сообщение об ошибке), точное достижение 100% (автоматическое изменение статуса).

Проверка корректности работы в разных браузерах:

Тестирование проводилось в следующих браузерах и версиях:

- Google Chrome – полная работоспособность, корректное отображение всех элементов, работа графиков, анимаций и localStorage.
- Mozilla Firefox – полная работоспособность, незначительные отличия в отрисовке теней (не критично).
- Microsoft Edge – полная работоспособность, идентично Chrome.

Заключение

В ходе выполнения курсового проекта была разработана интерактивная система «Трекер чтения книг», полностью соответствующая поставленным требованиям.

Результаты работы:

1. Проведён анализ предметной области, в результате которого выявлены ключевые сущности и процессы, необходимые для эффективного учёта чтения: книга, история чтения, дневная цель, достижения.
2. Выполнен сравнительный анализ существующих аналогов (Goodreads, Livelib), подтвердивший актуальность разработки собственного легковесного решения, ориентированного на мотивацию и детальный учёт прогресса.
3. Сформулированы функциональные требования и нефункциональные требования, которые полностью реализованы в разработанном приложении.
4. Разработана архитектура приложения с разделением на модули: constants.js, utils.js, data.js, bookManager.js. Такая структура обеспечивает удобство поддержки и дальнейшего развития.
5. Реализован полный CRUD-цикл для управления книгами: создание, чтение, обновление (прогресса и статуса), удаление.

6. Созданы алгоритмы расчёта статистики: серия непрерывного чтения (streak), средняя скорость чтения за 7 дней, суммарное количество страниц, количество завершённых книг.
7. Реализована система мотивации, включающая: установку и отслеживание дневной цели с визуализацией прогресса, систему достижений (три бейджа с автоматической разблокировкой), случайные мотивационные цитаты при каждой загрузке.
8. Обеспечена визуализация прогресса с помощью линейного графика, построенного на библиотеке Chart.js, отображающего количество прочитанных страниц за последние 7 дней.
9. Реализовано локальное хранение данных с использованием localStorage с автоматическим сохранением после каждого изменения и восстановлением при загрузке страницы.
10. Проведено тестирование на различных браузерах и устройствах, проверены все функциональные сценарии и граничные случаи.

Достигнутые цели:

- Приложение полностью клиентское, не требует сервера и постоянного подключения к интернету.
- Интерфейс интуитивно понятен, адаптирован под разные устройства (от мобильных телефонов до настольных мониторов).
- Пользователь получает визуальную обратную связь о своих действиях и мотивацию к регулярному чтению через систему целей и достижений.

Перспективы развития:

В будущем проект может быть расширен следующими возможностями:

- Добавление экспорта и импорта данных в форматах JSON для переноса между устройствами.
- Добавление возможности редактирования информации о книге (название, автор, количество страниц).

- Внедрение звуковых уведомлений при достижении дневной цели или разблокировке достижения.
- Расширение системы достижений (например, за прочтение 10, 50, 100 книг, за годовую серию чтения).
- Добавление страницы статистики с более детальными отчётами (месячные и годовые графики).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. docs.yandex: МАТЕРИАЛЫсПАР – первые шаги по разработке курсового проекта – URL: https://docs.yandex.ru/docs/view?url=ya-disk-public%3A%2F%2FqnQ%2F19LjAX2%2BLPuQfP8eQxJKtA3akI5TTB5eHlFU3ty5emyMuYVkXmen2G2hIUgRq%2FJ6bpmRyOJonT3VoXnDag%3D%3D%3A%2FМАТЕРИАЛЫсПАР.docx&name=МАТЕРИАЛЫсПАР.docx&nosw=1

2. GitHub: BookTracker – трекер чтения на чистом JS. – URL: https://github.com/algonacci/BookTracker

3. GitHub: Reading Tracker с графиками и статистикой. – URL: https://github.com/benjiwheeler/reading-tracker

4. Chart.js Documentation. – URL: https://www.chartjs.org/docs/latest/

5. GitHub: BookLog – ведение дневника читателя. – URL: https://github.com/Instrument/BookLog

6. GitHub: Simple Book Tracker на localStorage. – URL: https://github.com/ahmedhosny95/book-tracker

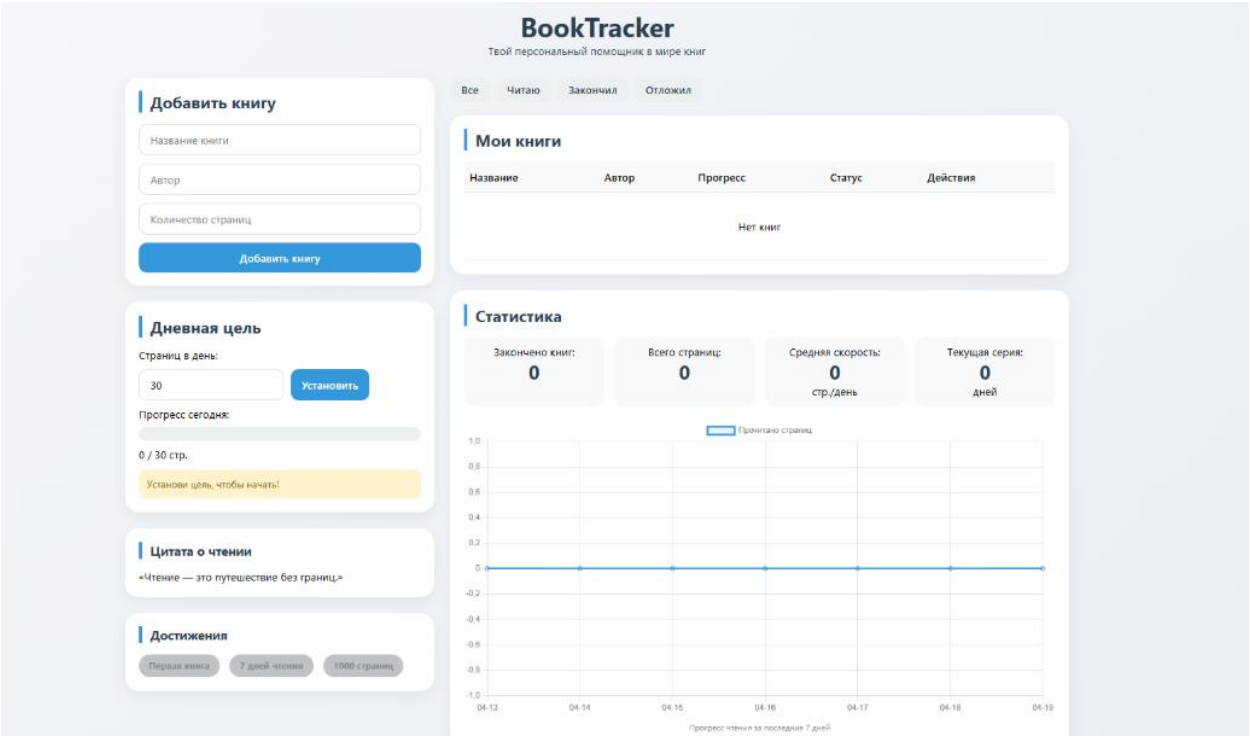
7. GitHub: Book Reading Progress Tracker. – URL: https://github.com/zadig/reading-tracker

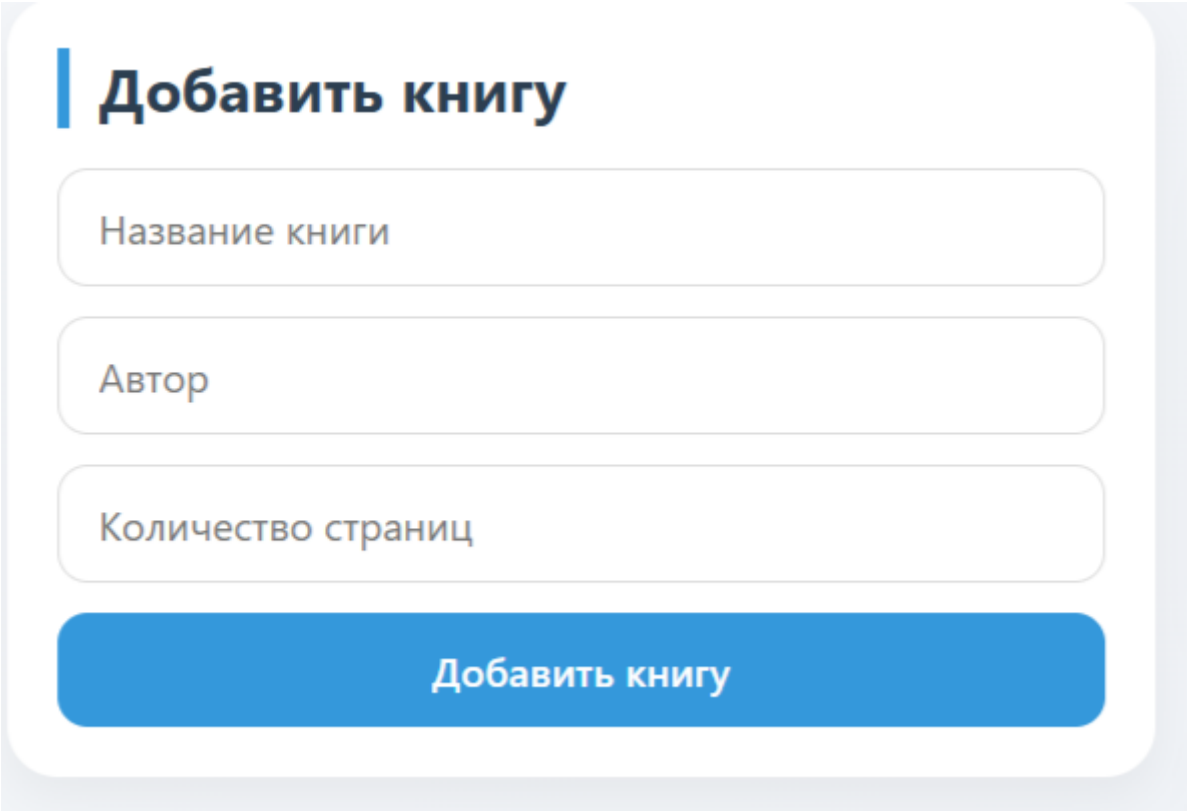
8. GitHub: Library Manager с отслеживанием прогресса. – URL: https://github.com/adarsh-chakravarty/Library-Manager

9. GitHub: BookShelf – трекер прочитанных книг. – URL: https://github.com/arieljake/bookshelf

10. GitHub: Reading Progress Log с диаграммами. – URL: https://github.com/ketanchoyal/reading-progress-log

Приложение


BookTracker
Твой персональный помощник в мире книг
Добавить книгу
Название книги
Автор
Количество страниц
Добавить книгу
Все
Читаю
Закончил
Отложил
Мои книги
Название
Автор
Прогресс
Статус
Действия
Нет книг
Дневная цель
Страниц в день:
30
Установить
Прогресс сегодня:
0 / 30 стр.
Установи цель, чтобы начать!
Статистика
Закончено книг:
0
Всего страниц:
0
Средняя скорость:
0
стр./день
Текущая серия:
0
дней
Прочитано страниц
1.0
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1.0
04-13
04-14
04-15
04-16
04-17
04-18
04-19
Прогресс чтения за последние 7 дней
Цитата о чтении
«Чтение — это путешествие без границ.»
Достижения
Первая книга
7 дней чтения
1000 страниц


Добавить книгу
Название книги
Автор
Количество страниц
Добавить книгу

Дневная цель

Страниц в день:

100 Установить

Прогресс сегодня:

0 / 100 стр.

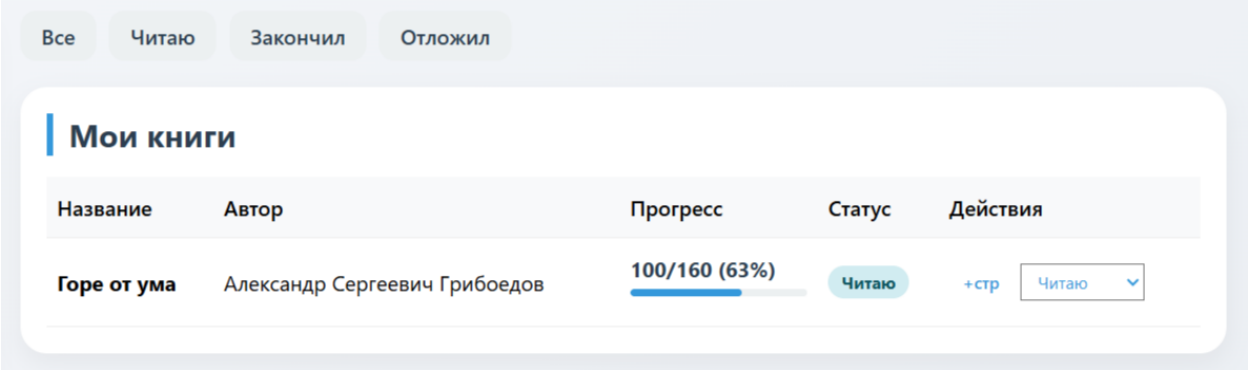
Дневная цель: 100 стр./день

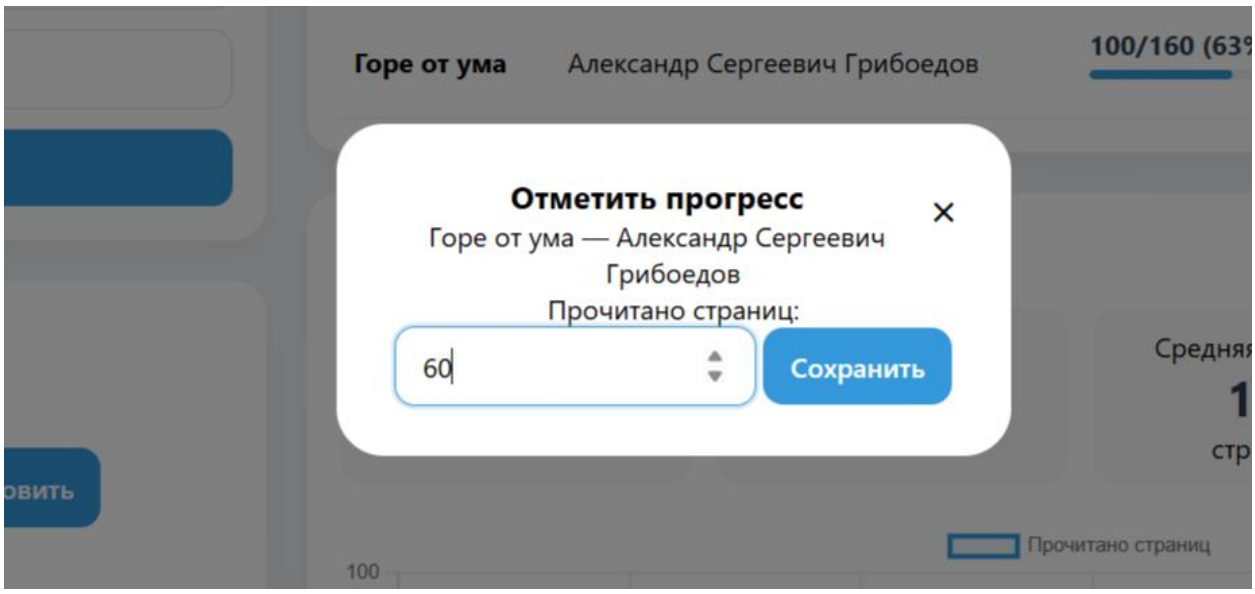
Цитата о чтении

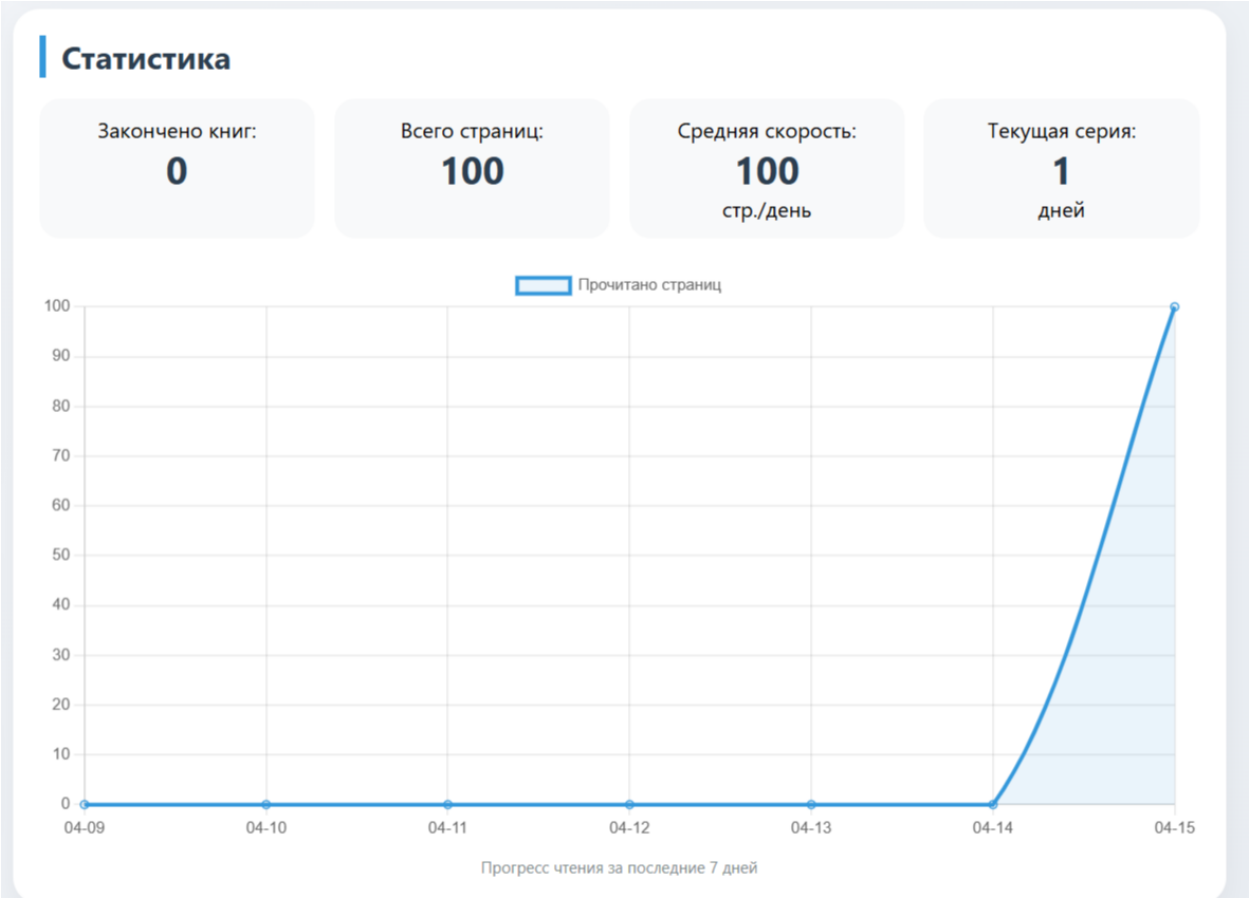
«В книгах — вся мудрость веков.»

Достижения

Первая книга | 7 дней чтения | 1000 страниц


Все
Читаю
Закончил
Отложил
Мои книги
Название
Автор
Прогресс
Статус
Действия
Горе от ума
Александр Сергеевич Грибоедов
100/160 (63%)
Читаю
+стр
Читаю


Горе от ума
Александр Сергеевич Грибоедов
100/160 (63
Отметить прогресс
Горе от ума — Александр Сергеевич Грибоедов
Прочитано страниц:
60
Сохранить
Средняя
1
стр
овить
Прочитано страниц
100


Статистика
Закончено книг:
0
Всего страниц:
100
Средняя скорость:
100
стр./день
Текущая серия:
1
дней
Прочитано страниц
100
90
80
70
60
50
40
30
20
10
0
04-09
04-10
04-11
04-12
04-13
04-14
04-15
Прогресс чтения за последние 7 дней